DOSSIER DE FABRICATION : Prototype de Minuteur d'œuf

Membres du groupe : ZAHRI Ayoub
COULIBALY Mohamed
TOUSSIDO Simo

Nom des encadrants : Mme André
M.Dubois
M.Dussart
Mme.Ringot

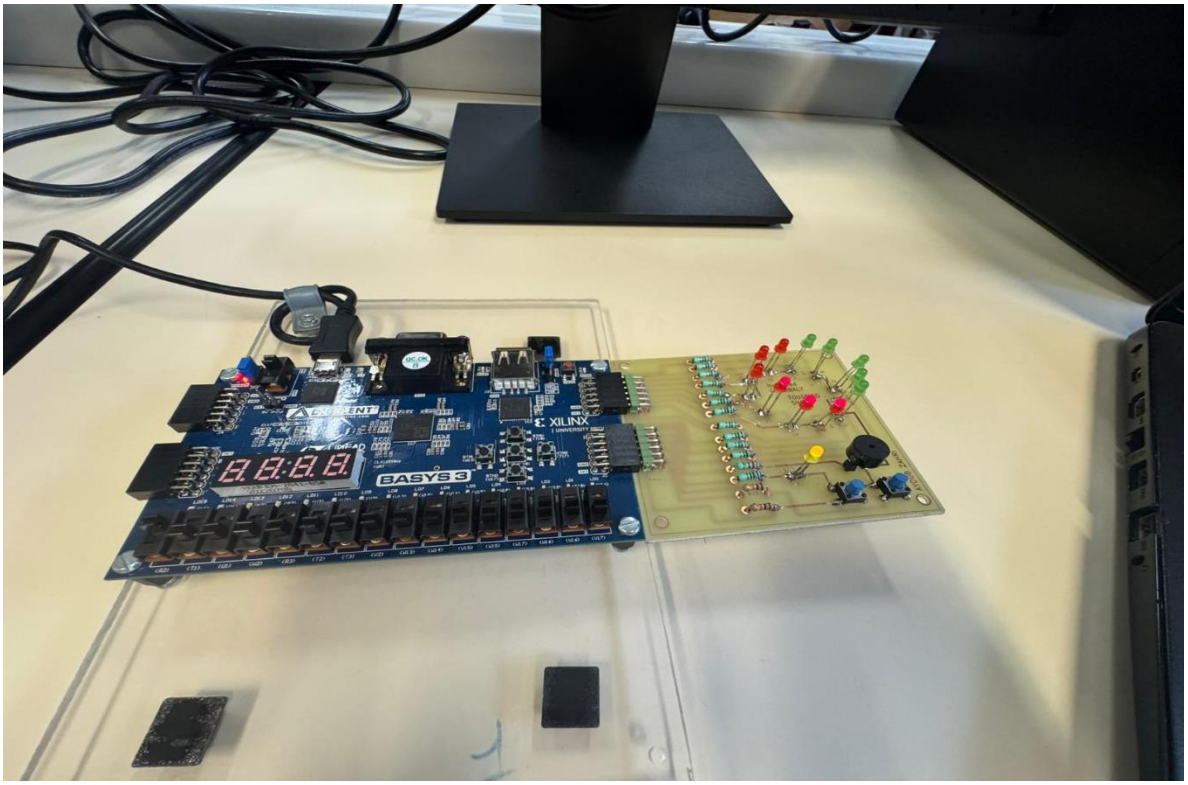


Table des matières :

Introduction - - -

INTRODUCTION:

Dans le cadre de notre formation en génie électrique et informatique industrielle, un projet nous a été proposé pour mettre en pratique nos compétences. Ce projet consiste à concevoir et vérifier la partie électronique et informatique d'un système. Nous devons répondre à un cahier des charges qui demande de réaliser un prototype de minuteur pour la cuisson d'œufs.

Ce dossier de fabrication explique les différentes étapes qui nous ont permis de réaliser ce prototype. Nous expliquerons le choix des matériels ainsi que les difficultés que nous avons rencontré lors de la réalisation du projet.

I) Le cahier des charges et la planification du travail

1) présentation du cahier des charges

Ce cahier des charges nous a été fourni par notre « manager », qui nous a demandé de concevoir un prototype fonctionnel.

Le système demandé doit fonctionner de la manière suivante :

- Il doit être composé de deux boutons poussoirs : l'un permettant la mise en marche du minuteur, et l'autre permettant l'ajout de temps.
- Il doit comporter six LED rouges qui s'éteignent toutes les 30 secondes, ainsi que cinq LED vertes qui s'éteignent toutes les 15 secondes.
- Pendant toute la durée du cycle, une LED doit clignoter afin d'indiquer le fonctionnement du système.
- À la fin du temps imparti, un buzzer doit émettre un signal sonore.

a) besoins

Afin de mener à bien notre projet, il est nécessaire d'identifier nos besoins pour concevoir un prototype fonctionnel. Dans notre cas, ces besoins concernent principalement :

- La programmation en langage VHDL ;
- La conception du schéma de câblage et du routage à l'aide du logiciel KiCad ;
- Les différentes étapes de réalisation d'une carte électronique, de l'impression jusqu'à la soudure ;

- Les méthodes de test et de vérification d'une carte électronique.

Il est également nécessaire d'améliorer notre organisation de travail afin de respecter la contrainte de temps imposée par le projet.

b) Objectif du projet

L'objectif principal de notre projet est de concevoir un prototype de minuteur destiné à la cuisson des œufs, en respectant les consignes définies dans le cahier des charges.

c) ressources

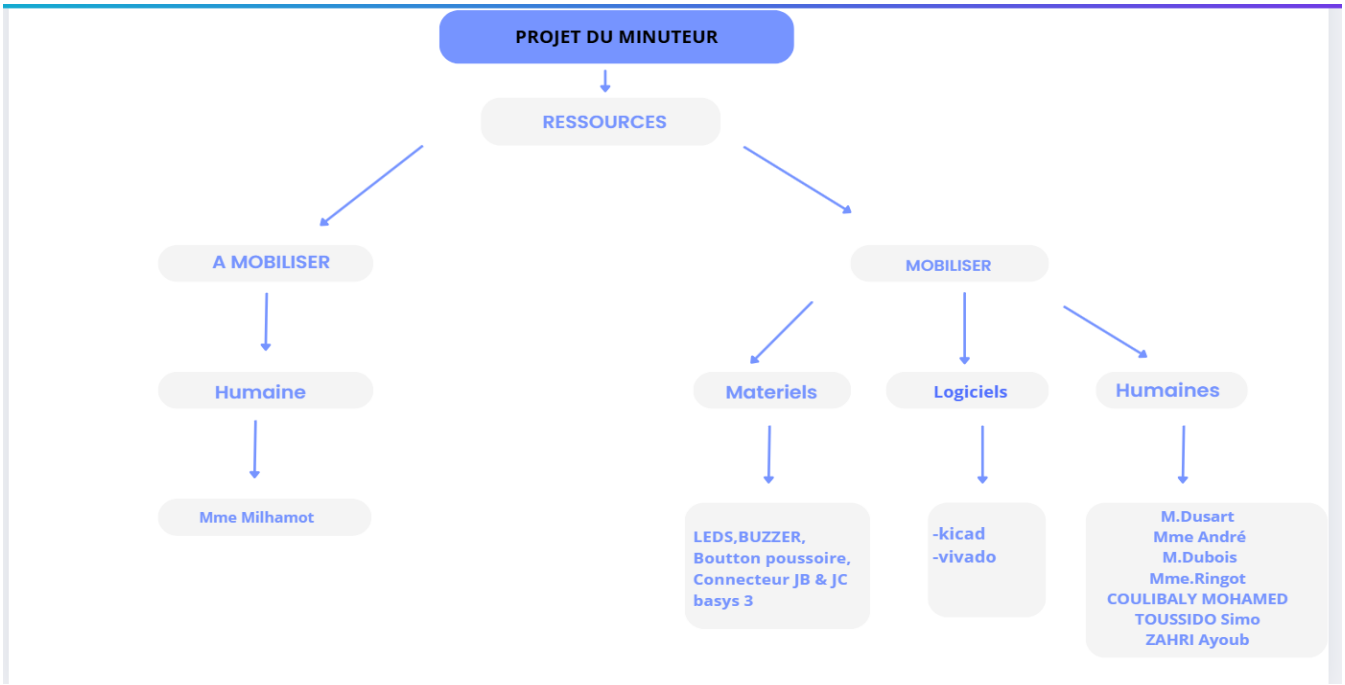


Figure 1 : Organigramme ressource

2) Planification du projet

La planification de notre projet se tient sur cinq semaines. Code de couleurs :

Tableau de planification des tâches pour le projet du minuteur d'œuf

Travail effectuer en trinôme

Jores cabrel TOUSSIDO SIMO

Ayoub ZAHRI

Mohamed Coulibaly

Semaines / **Tâches**	Semaine 1	Semaine 2	Semaine 3	Semaine 4	Semaine 5
Analyse du cahier des charges					
Recherches sur les logiciels					
Recherches sur les composants					
Concevoir le schéma technique					
Concevoir le programme					
Vérifier le programme					
Routage de la carte					
Gravure de la carte					
Vérifications de la continuité de la carte					
Soudure des composants					
Vérification de la carte					
Vérification de la fonctionnalité du prototype					
Préparation d'oral de compétence					
Rédaction du dossier de conception					

II) Conception du système

1) a) Analyse fonctionnelle et systémique

Le fonctionnement de notre système est basé sur le cahier des charges. Le comportement attendu est le suivant :

- À l'état initial, une impulsion sur le bouton poussoir numéro 1 permet de démarrer le compte à rebours à un temps prédéfini de 3 minutes, correspondant aux six LED rouges.
- Des appuis successifs sur le bouton poussoir numéro 2 permettent d'ajouter quinze secondes au temps de cuisson.
- Une LED jaune clignote une fois par seconde pendant tout le processus pour indiquer que le minuteur fonctionne.
- Le temps maximum de cuisson est de 4 minutes 15 secondes. Les LED rouges s'éteignent toutes les 30 secondes et les LED vertes toutes les 15 secondes.
- À la fin du décompte, toutes les LED clignotent et le buzzer retentit. Un nouvel appui sur le bouton poussoir numéro 1 permet de revenir à l'état initial.

b) schéma d'implantation

Pour l'implantation, notre système se décompose en deux parties : informatique et électronique.

La partie informatique

Dans cette partie, nous allons écrire le fonctionnement du système décrit précédemment en langage VHDL (Very High-Speed Integrated Circuits Hardware Description Language) à l'aide du logiciel Vivado. Cet outil nous permettra de :

- Rédiger le code,
- Vérifier sa syntaxe,
- Simuler le fonctionnement du circuit,
- Tester le comportement attendu du minuteur.

La partie électronique

Pour la partie électronique, nous allons utiliser KiCad pour réaliser deux types de schémas : le schéma de câblage et le schéma de routage**.**

- Le schéma de câblage représente la partie graphique du circuit électronique et les connexions entre les composants.
- Le schéma de routage consiste à disposer les composants sur la carte et à tracer les pistes de manière optimale. Cette étape est importante, car elle servira pour l'impression de la carte sur laquelle nous souderons ensuite nos composants.

Le tableau ci-dessous présente la liste des composants utilisés et leurs empreintes pour la réalisation du schéma :

Référence	Composant	Librairie empreinte	Empreinte
J1,2	Conn_02x06_Odd_Even	Connector_PinHeader_2.54mm	PinHeader_2x06_P2.54mm_Vertical
R1….15	R_US	Resistor_THT	R_Axial_DIN0207_L6.3mm_D2.5mm_P7.62mm_Horizontal
D1….12	LED	LED_THT	LED_D3.0mm
BZ1	Buzzer	Connector_Samtec_HPM_THT	Samtec_HPM-02-01-x-S_Straight_1x02_Pitch5.08mm
SW1,2	SW_Push_Dual	Button_Switch_THT	SW_PUSH_6mm_H5mm

2) Implantation et choix matériels (liste des composants et de leurs caractéristique, justification des choix)

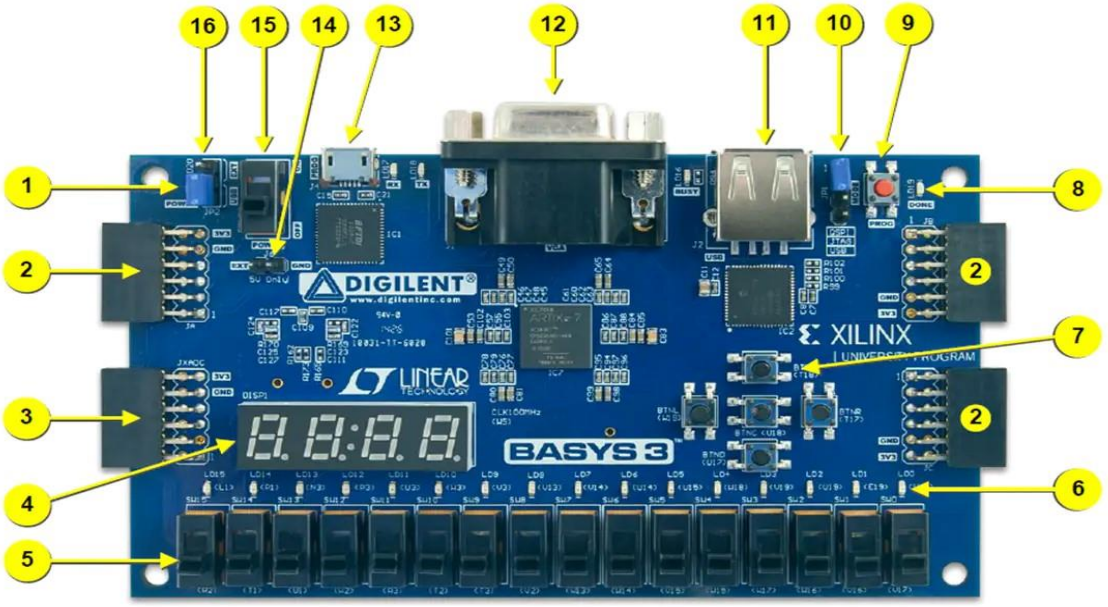


Figure 1. Basys 3 FPGA board with callouts.

Callout	Component Description	Callout	Component Description
1	Power good LED	9	FPGA configuration reset button
2	Pmod port(s)	10	Programming mode jumper
3	Analog signal Pmod port (XADC)	11	USB host connector
4	Four digit 7-segment display	12	VGA connector
5	Slide switches (16)	13	Shared UART/ JTAG USB port
6	LEDs (16)	14	External power connector
7	Pushbuttons (5)	15	Power Switch
8	FPGA programming done LED	16	Power Select Jumper

Figure 2 : Carte Basys3

Pour notre projet, nous avons utilisé la carte FPGA Basys3. Nous l'avons choisie car elle permet de réaliser un prototype sans construire une carte électronique complète.

La Basys3 possède plusieurs éléments utiles pour notre projet :

- 16 LED pour afficher le temps restant et indiquer le fonctionnement du système,
- 5 boutons poussoirs pour démarrer/arrêter le minuteur et ajouter du temps,
- 16 interrupteurs, qui peuvent également être utilisés pour le démarrage et l'ajout de temps,
- 1 FPGA Artix-7 qui permet de programmer toutes les fonctions du minuteur en VHDL,

- Les connecteurs JB et JC**,** utilisés pour relier la carte Basys3 à notre carte électronique.

Ces caractéristiques nous ont permis de tester toutes les fonctionnalités du minuteur directement sur la carte Basys 3.

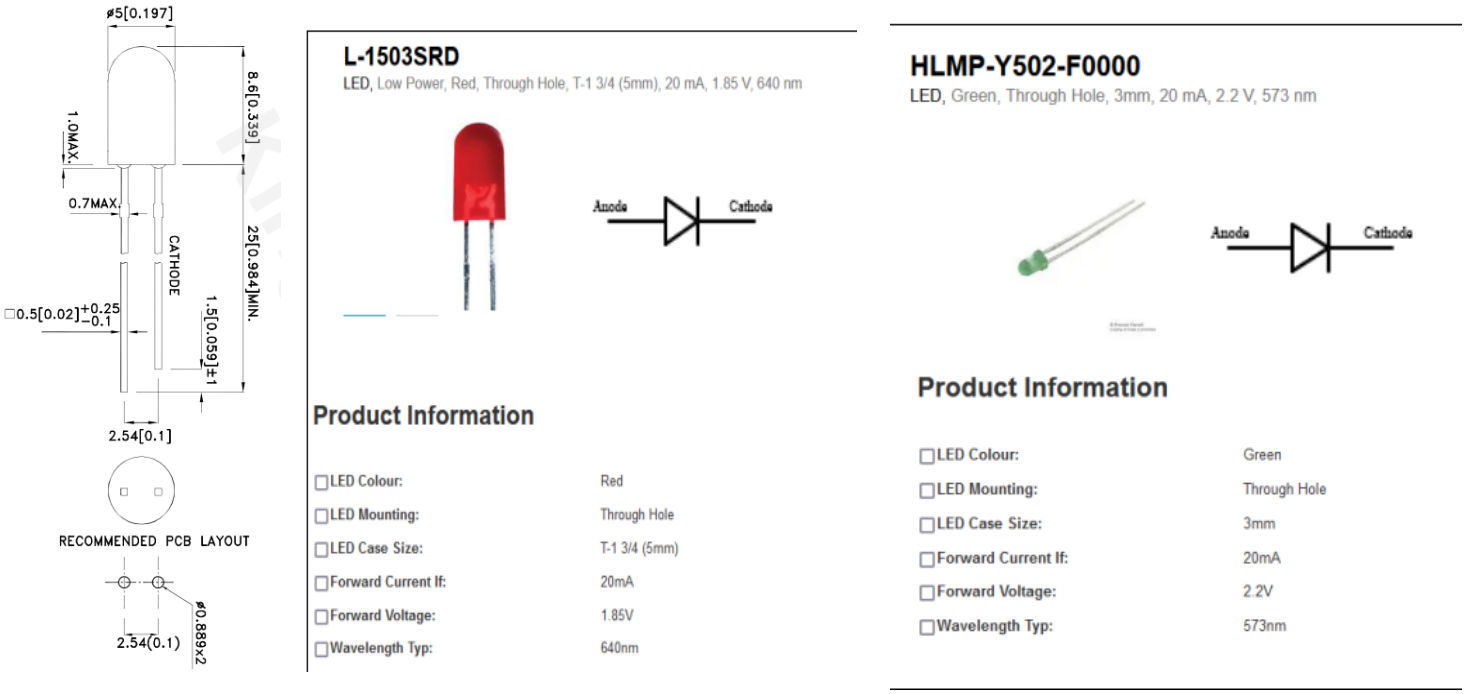


Figure 3 : LED

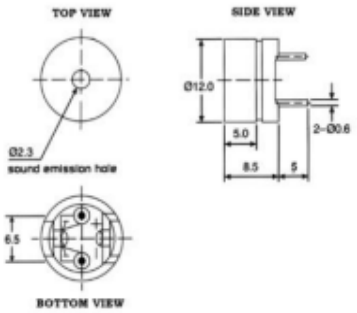
Les LED rouges et vertes sont composées d'une anode, reliée à l'alimentation, et d'une cathode**,** reliée à la masse. Elles permettent de visualiser le fonctionnement du prototype et d'indiquer le temps restant pendant la cuisson.

41.T70P015H-LF

Magnetic Transducer - 1.5V Pin Noryl Lead Free Material (RoHS) - Color - Black



Dimension (mm)



Product Information

Model Number	41.T70P015H-LF
Operating Voltage Range (V)	1.0 ~ 5.0
Rated Voltage (V)	1.5
Max. Rated Current+ (mA)	30
Coil Resistance (Ω)	16.0 +/- 4.5
Coil Impedance (Ω)	40
Min. Sound Output at 10cm+ (dBA)	85
Resonant Frequency (Hz)	2048
Operating Temperature (°C)	-20 ~+60
Terminal	Pin
Housing Material	Noryl
Weight (g)	2

+ Value applying rated voltage reasonant frequency, 1/2 duty, square wave

* Value applying reasonant frequency, 1/2 duty, square wave, measuring current 60uA

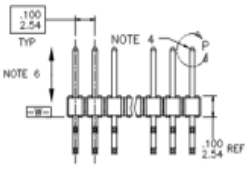
Figure 4 : Buzzer

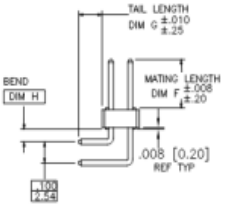
Le buzzer utilisé dans notre prototype est un transducteur électromagnétique qui transforme le signal électrique en vibrations mécaniques, ce qui produit un son. Il fonctionne avec la fréquence du signal qu'on lui envoie et a pour rôle de signaler la fin du temps de cuisson par un signal sonore.

10129382-906004BLF

Pin Header, Board-to-Board, 2.54 mm, 2 Rows, 6 Contacts, Through Hole Right Angle, FCI Econostik







Product Information

Connector Systems	Board-to-Board	Pitch Spacing	2.54mm
No. of Rows	2Rows	No. of Contacts	6Contacts
Contact Termination Type	Through Hole Right Angle	Product Range	FCI Econostik
Connector Shroud	Unshrouded	Contact Material	Copper Alloy
Contact Plating	Gold Plated Contacts	Connector Type	Pin Header
SVHC	No SVHC (12-Jan-2017)		

Figure 5 : Connecteur Pmod JB & JC

Les connecteurs Pmod JB et JC nous permettent de connecter notre carte électronique au Basys 3. Ils sont composés de 12 broches et permettent de faire la liaison entre la carte Basys 3 et les différents composants de notre prototype (LED, boutons poussoirs, buzzer, etc.).

Caractéristiques

- **Puissance: 0.6 W**
- Température Maximale: 155°C
- Température Minimale: -55°C
- Montage: Through Hole
- Diamètre: 2.5mm
- Longueur: 6.8mm
- Technologie: Carbon Film
- Coéfficient de température: -1200 ⟶ 0ppm/°C
- Type de raccord: Axial
- Tolérance: 5%



Figure 6 : Résistances

Les résistances sont utilisées avec les LED, les boutons poussoirs et le buzzer. Elles servent à limiter le courant qui traverse ces composants afin de les protéger et de garantir leur bon fonctionnement, en évitant de dépasser le courant maximal qu'ils peuvent supporter.



Specification:
Terminals Number: 4
Temperature: -25~70C
Withstand voltage: AC250V
Rated load: DC12V 50mA
Contact resistance: less than 0.03 Ohm
Insulation resistance: More than 100M Ohm
Dimension: 6x6x7.5mm/0.24*0.24*0.3inch(L*W*H)
Weight: 3g

Figure 7 : Bouton poussoir

Les boutons servent à commander le minuteur. Au repos, leur contact est ouvert et il se ferme lorsqu'on appuie dessus. Chaque bouton est relié à la masse et à l'alimentation avec une résistance, ce qui envoie un signal à la carte quand le bouton est appuyé.

3) Programmation (Description fonctionnelle)

Pour la programmation, nous allons utiliser le langage VHDL comme nous l'avons souligné précédemment. Ce code sera scindé en plusieurs parties (process).

Le code VHDL est divisé en deux grandes parties : l'entité et l'architecture.

- L'entité définit les entrées et sorties, c'est-à-dire les composants utilisés. Elle est composée également de bibliothèques nécessaires au programme. Pour ce projet, nous avons utilisé trois bibliothèques :
  - **library IEEE** : bibliothèque principale qui permet d'inclure les autres bibliothèques,
  - **IEEE.STD_LOGIC_1164.ALL** : pour utiliser les types logiques standard en électronique numérique,
  - **IEEE.NUMERIC_STD.ALL** : pour effectuer des calculs arithmétiques.

- L'architecture décrit le fonctionnement global du système à l'aide de signaux et de process. Nous avons créé pour notre code cinq processus :

1. Le premier gère l'antirebond du bouton poussoir 1,
2. Le second gère l'antirebond du bouton poussoir 2,
3. Le troisième décrit le fonctionnement principal du minuteur,
4. Le quatrième et le cinquième gèrent le clignotement final des LED et le buzzer**.**

Et à la fin de notre architecture nous décrirons l'état de nous sorti.

III) Réalisation du prototype

1) Différentes étapes

Les étapes qui ont permis de réaliser notre prototype fonctionnel sont les suivantes :

- Analyser et comprendre le cahier des charges,
- Identifier et comprendre les datasheets des différents composants,
- Écrire et simuler le programme en langage VHDL,
- Réaliser le schéma de câblage et de routage,
- Imprimer et percer la carte,
- Souder les composants sur la carte,
- Effectuer les tests de continuité,
- Implémenter et tester notre programme sur la carte.



Figure 8 : fer à souder

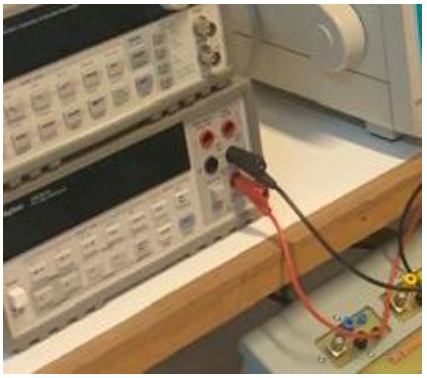


Figure 9 : Appareil de mesure

2) Tests problèmes et solutions

Le test

Pour tester notre carte, nous avons d'abord utilisé le schéma pour effectuer un test de continuité. Ensuite, nous avons implanté notre programme pour observer son fonctionnement. Le programme a également été testé par simulation dans Vivado**.**

Les problèmes

Lors de la réalisation du projet, nous avons rencontré plusieurs problèmes :

- Certaines LED étaient inversées, donc elles ne s'allumaient pas lorsque le programme était implanté.
- Une résistance défectueuse empêchait l'allumage d'une LED.
- Une piste de terre sur la carte avait une mauvaise continuité, ce qui empêchait certaines LED de s'allumer.
- Une erreur dans le schéma Kicad **:** l'alimentation des boutons poussoirs avait été branchée sur le mauvais pin du connecteur JC.

Les solutions

Pour résoudre ces problèmes, nous avons procédé étape par étape :

- Nous avons testé les LED avec une alimentation externe pour vérifier leur fonctionnement sans le programme.
- Nous avons contrôlé chaque piste de la carte avec un test de continuité et réparé celles qui posaient problème.
- Nous avons vérifié le sens des LED et replacé celles qui étaient inversées.
- Enfin, nous avons mesuré les résistances et remplacé celle qui était défectueuse.

Cette méthode nous a permis de corriger tous les problèmes et d'avoir un prototype fonctionnel.

3) Améliorations possibles

Pour améliorer le prototype, on aurait pu :

- Ajouter un bouton pour choisir le temps de cuisson selon le type d'œuf,
- Faire que le buzzer s'arrête tout seul après un certain temps,
- Permettre de réinitialiser le minuteur avec un double appui ou en maintenant le bouton start pendant 5 secondes.
- Programmer l'afficheur segment de la Basys 3 pour afficher le temps.

Conclusion:

En conclusion, ce projet nous a permis de concevoir et vérifier un prototype fonctionnel en respectant le cahier des charges. Malgré quelques difficultés rencontrées, nous avons réussi à mettre en pratique nos compétences en électronique et en programmation. Ce travail nous a aussi appris à identifier les problèmes, tester et trouver des solutions**,** ce qui renforce notre capacité à concevoir et vérifier un système.

Bibliographie: lien carte Bibliographie

• Datasheets des composants : LED, résistances, buzzer, boutons poussoirs (disponibles sur Moodle).

• Basys3 Reference Manual (disponible sur Moodle).

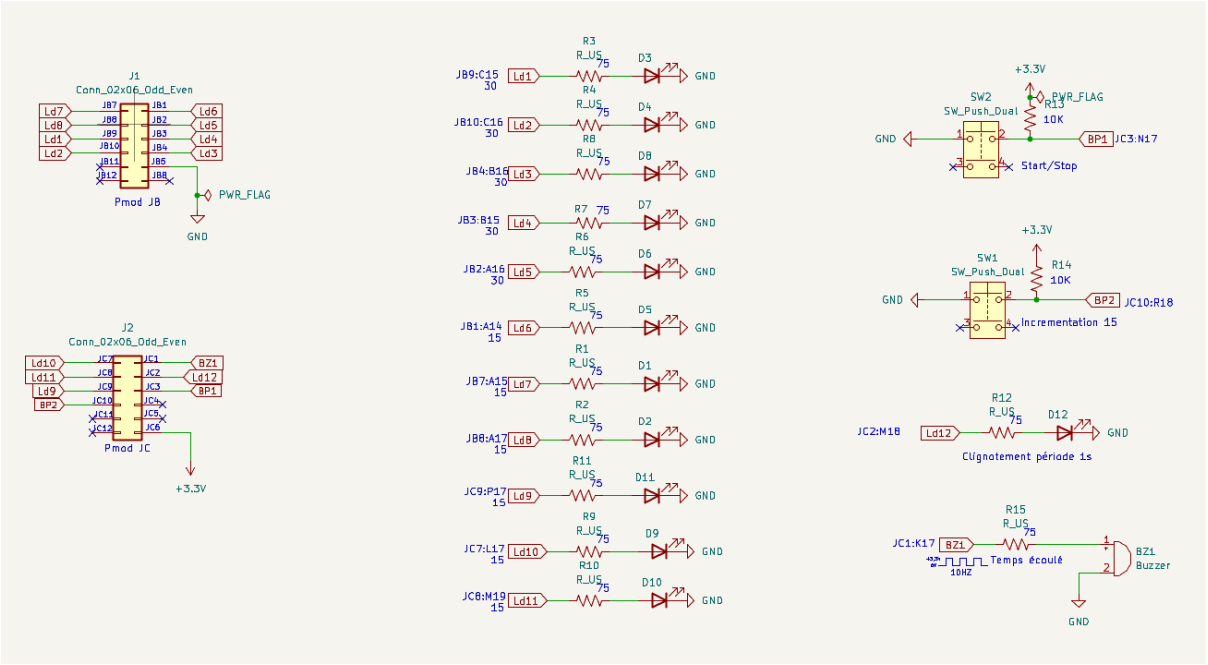
• Vivado – Guide d'implantation de programme VHDL (disponible sur Moodle).

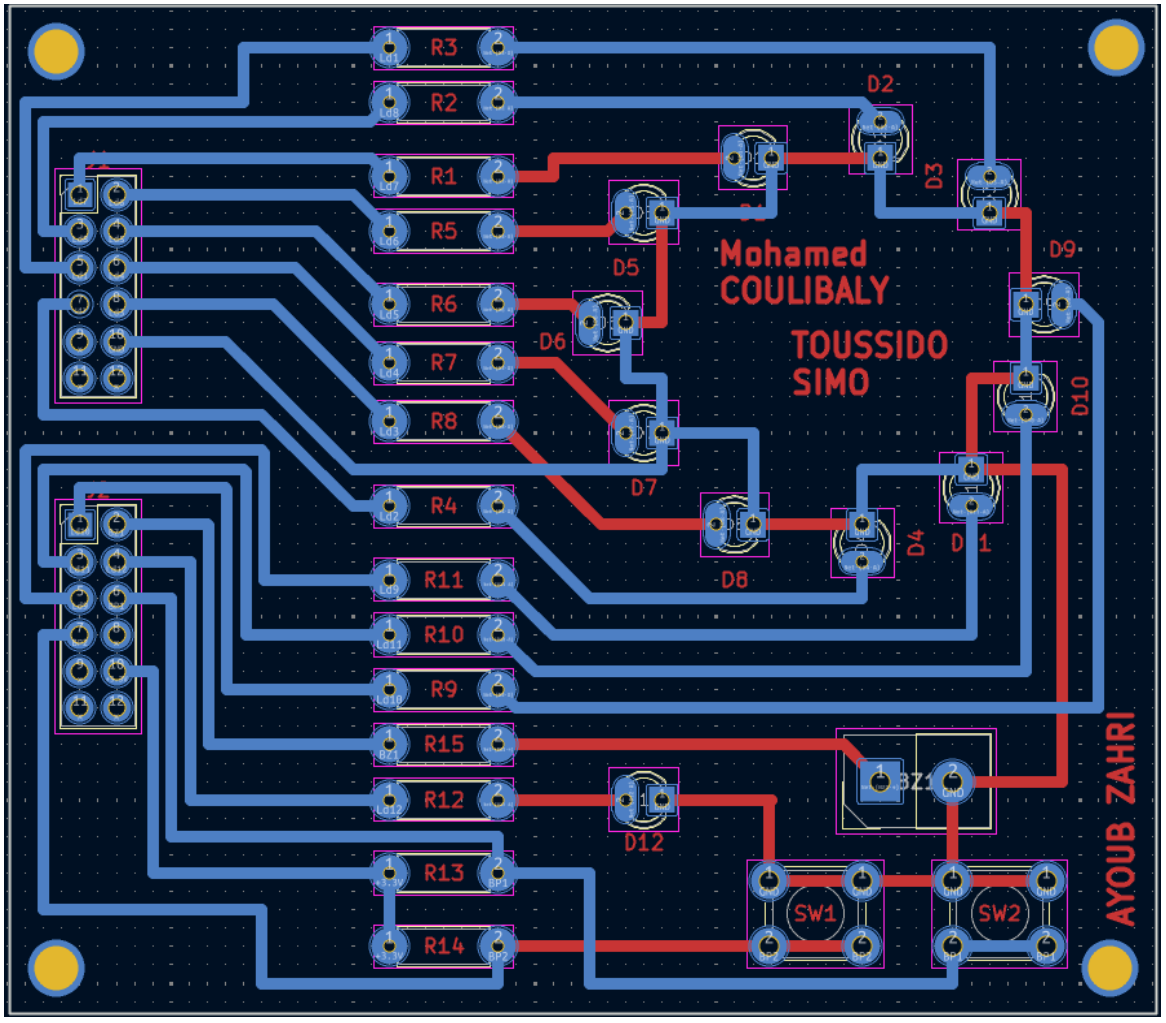
• KiCad – Guide d'utilisation (disponible sur Moodle).

figure	description	page
1	Organigramme ressources	9
2	Carte Basys 3	11
3	LED	12
4	Buzzer	13
5	Connecteur Pmod JB & JC	13
6	Résistances	14
7	Bouton poussoir	14
8	Fer à Souder	15
9	Appareil de mesure	15
10	fer à souder	17

Annexes:

- schéma:





- Programme:

```
31  library IEEE;
32  use IEEE.STD_LOGIC_1164.ALL;
33  use IEEE.NUMERIC_STD.ALL;
34
35  entity E_SAE_PROJECT is
36      Port (
37          H     : in  std_logic;                      -- horloge 100 MHz
38          BP1   : in  std_logic;                      -- bouton start/reset
39          BP2   : in  std_logic;                      -- bouton ajout cpt
40          L     : out std_logic;
41          LV    : out std_logic_vector(5 downto 0);
42          LR    : out std_logic_vector(4 downto 0);
43          buzz : out std_logic
44      );
45  end E_SAE_PROJECT;
46  architecture A_SAE_PROJECT of E_SAE_PROJECT is
47
48      -- États de la machine
49      signal Q           : std_logic_vector(1 downto 0) := "00";
50
51      -- Compteurs
52      signal count_cpt   : integer range 0 to 100000000 := 0; -- horloge 100MHz → 1 sec
53      signal cpt         : integer range 0 to 255 := 180;
54      signal Ltemp       : std_logic := '0';
55      signal countLVR    : integer range 0 to 25000000 := 0;
56      signal LVR         : std_logic := '0';
57     signal count_Ltemp : integer range 0 to 50000000 := 0;
58      -- Anti-rebond BP1
59      signal BP_filtre   : std_logic := '0';
60      signal BP_mem      : std_logic := '0';
61      signal antiB_cnt   : integer range 0 to 100000 := 0;
62      signal BP1_last    : std_logic := '0';
63
64      -- Anti-rebond BP2
65      signal BP2_filtre : std_logic := '0';
66      signal BP2_mem     : std_logic := '0';
67      signal antiB2_cnt : integer range 0 to 100000 := 0;
68      signal BP2_last    : std_logic := '0';
69      --BUZZER
70      signal buzz_count : integer range 0 to 499999 := 0;
71      signal buzz_10hz   : std_logic := '0';
74
75  begin
76     -------------------------------
77      -- Anti-rebond BP1
78      -------------------------------
79      process(H)
80      begin
81          if rising_edge(H) then
82              if BP1 = BP_mem then
83                  if antiB_cnt < 100000 then
84                      antiB_cnt <= antiB_cnt + 1;
85                  end if;
86              else
87                  antiB_cnt <= 0;
88              end if;
89              BP_mem <= BP1;
90              if antiB_cnt = 100000 then
91                  BP_filtre <= BP1;
92              end if;
93          end if;
94      end process;
95
96      -------------------------------
97      -- Anti-rebond BP2
98      -------------------------------
99      process(H)
100     begin
101         if rising_edge(H) then
102             if BP2 = BP2_mem then
103                 if antiB2_cnt < 100000 then
104                     antiB2_cnt <= antiB2_cnt + 1;
105                 end if;
106             else
107                 antiB2_cnt <= 0;
108             end if;
109             BP2_mem <= BP2;
110             if antiB2_cnt = 100000 then
111                 BP2_filtre <= BP2;
112             end if;
113         end if;
```

```
    ------------------------------
    -- Machine à états principale
    ------------------------------
    process(H)
    begin
        if rising_edge(H) then
         if BP2_filtre = '0' and BP2_last = '1' then
                    if cpt <= 240 then
                        cpt <= cpt + 15;
                    else
                        cpt <= 255;
                    end if;
                end if;
                BP2_last <= BP2_filtre;
          if Q = "00" then

                count_cpt <= 0;
                count_Ltemp <= 0;
                Ltemp <= '0';

                if BP_filtre = '0' and BP1_last = '1' then
                    Q <= "01";
                end if;

            elsif Q = "01" then
                -- Décompte 1 seconde
                if cpt > 0 then
                    if count_cpt < 100000000 then
                        count_cpt <= count_cpt + 1;
                    else
                        count_cpt <= 0;
                        cpt <= cpt - 1;  -- décrément toutes les secondes
                    end if;

                    -- Clignotement rapide LED
                    if count_Ltemp < 50000000 then
                        count_Ltemp <= count_Ltemp + 1;
                    else
                        count_Ltemp <= 0;
                        Ltemp <= not Ltemp;
                end if;
            else
                Q <= "10";  -- passage à clignotement final
            end if;

        elsif Q = "10" then
            -- Attente BP1 pour réinitialiser le cycle
            if BP_filtre = '0' and BP1_last = '1' then
                Q <= "00";
                cpt <= 180;
            end if;
        end if;

        BP1_last <= BP_filtre;
    end if;
end process;

------------------------------
-- Clignotement final (LEDs rouges et vertes)
------------------------------
process(H)
begin
    if rising_edge(H) then
        if Q = "10" then
            if countLVR < 25000000 then
                countLVR <= countLVR + 1;
            else
                countLVR <= 0;
                LVR <= not LVR;  -- clignotement final visible
            end if;
        else
            countLVR <= 0;
        end if;
    end if;
end process;
```

```
191
192      ------------------------------
193     -- BUZZER
194     ------------------------------
195     process(H)
196     begin
197        if rising_edge(H) then
198           if Q = "10" then
199              if buzz_count < 49999 then
200                 buzz_count <= buzz_count + 1;
201              else
202                 buzz_count <= 0;
203                 buzz_10hz  <= not buzz_10hz;
204              end if;
205           else
206              buzz_count <= 0;
207              buzz_10hz  <= '0';
208           end if;
209        end if;
210     end process;
211
212
213
214     ------------------------------
215       -- BUZZER
216       --------------------------------
217
218         buzz <= buzz_10hz when Q="10" else '0';
219
220       --------------------------------
221       -- LED principale
222       --------------------------------
223       L <= Ltemp when Q = "01" else
224            LVR when Q = "10" else
225            '0';
226
227       --------------------------------
228       -- LEDs rouges
229       --------------------------------
230       LV(0) <= LVR when Q="10" else
231                '1' when (Q="01" and cpt>0) or (Q="00") else '0';
232       LV(1) <= LVR when Q="10" else
233                '1' when (Q="01" and cpt>30) or (Q="00") else '0';
234       LV(2) <= LVR when Q="10" else
235                '1' when (Q="01" and cpt>60) or (Q="00") else '0';
236       LV(3) <= LVR when Q="10" else
237                '1' when (Q="01" and cpt>90) or (Q="00") else '0';
238       LV(4) <= LVR when Q="10" else
239                '1' when (Q="01" and cpt>120) or (Q="00") else '0';
240       LV(5) <= LVR when Q="10" else
241                '1' when (Q="01" and cpt>150) or (Q="00") else '0';
242       --------------------------------
243       -- LEDs vertes
244       --------------------------------
245       LR(0) <= LVR when Q="10" else
246                '1' when (Q="00" and cpt>180) or (Q="01" and cpt> 180) else '0';
247       LR(1) <= LVR when Q="10" else
248                '1' when (Q="00" and cpt>195) or (Q="01" and cpt>195) else '0';
249       LR(2) <= LVR when Q="10" else
250                '1' when (Q="00" and cpt>210) or (Q="01" and cpt>210) else '0';
251       LR(3) <= LVR when Q="10" else
252                '1' when (Q="00" and cpt>225) or (Q="01" and cpt >225) else '0';
253       LR(4) <= LVR when Q="10" else
254                '1' when (Q="00" and cpt >240 ) or (Q="01" and cpt>240) else '0';
255     end A SAE PROJECT;
```